

XRPL Developer Experience Feedback

Technical Context

Project: Raise

Environment: XRPL Lending Hackathon Devnet, Network ID 4001

Client library Version: xrp1.js 5.2.0

Flavour: Loaded (XLS-65 + XLS-66 + XLS-56 Batch)

Node: 24.21.0

Chosen Track: Track 1 (Open-ended vault, Lending Protocol V1 target)

Raise provides secondary liquidity for lending-vault shares: investors can transfer vault shares while the underlying loan continues. We validated vault creation, deposits, broker setup, share transfer, receiver authorization, redemption rights, and atomic XRP-for-share settlement. The documentation explained the vault, broker and loan separately, but did not clearly show how they were connected. We had to follow the vaultID and the LoanbrokerID across several ledger objects to verify the full relationship.

Our main takeaway was that the ledger guardrails generally worked as expected, but some prerequisites and result states only became clear after a failed submission or after we inspected the raw ledger data. Indeed, the available liquidity was visible, but the position value and the utilisation had to be calculated from different ledger fields. The main gap was not missing functionality. It was how much protocol knowledge we needed to check readiness and interpret the result safely.

1. Receiver authorization is not obvious for transferable MPTs

Field	Details
Category / Severity	Documentation & Tutorials / UX · High
Goal	Transfer vault-share MPTs to a new investor.
Expected	Transfer succeeds when CanTransfer=true and RequireIssuerAuth=false.
Actual	Transfer failed with tecNO_AUTH until the receiver submitted MPTokenAuthorize.
Reproduction	Unopted receiver → tecNO_AUTH → authorize → retry → tesSUCCESS
Impact	We discovered the receiver opt-in requirement only after the transfer failed.
Improvement	Document the opt-in step and add a receiver-readiness check.
Evidence	EC138D...6A27 · AC3B4A...2044 · 84C763...2350

We initially assumed that `CanTransfer=true` and `RequireIssuerAuth=false` made the shares freely transferable. When the transaction failed, we could not immediately tell whether the issue was the issuer settings, the receiver account, or our transaction structure. Only after trying `MPTokenAuthorize` did we understand that receiver opt-in was a separate requirement. The behaviour made sense afterwards, but we learned it from the failed transaction rather than the documentation.

2. Batch `tesSUCCESS` does not clearly indicate economic completion

Field	Details
Category / Severity	Client Library / UX · High
Goal	Atomically exchange XRP for vault-share MPTs using an all-or-nothing Batch.
Expected	The result clearly shows whether the exchange completed.
Actual	The outer Batch returned <code>tesSUCCESS</code> , but neither transaction was applied.
Reproduction	Unfunded XRP leg → submit Batch → outer <code>tesSUCCESS</code> → verify that neither leg was applied
Impact	An application could incorrectly show a rolled-back trade as completed.
Improvement	Expose aggregate Batch status and document the inner metadata that callers must inspect.
Evidence	Success <code>4D4686EA...0F9</code> : 1,000,000 drops ↔ 450,000 shares, fee 60 drops · Failure <code>EC8802E7...813</code> : neither leg applied, fee 12 drops

We deliberately made the XRP payment fail to check whether the Batch would roll back atomically. The rollback worked, but the outer `tesSUCCESS` initially made us think the exchange had completed. We only confirmed the real outcome after checking the inner results and comparing the XRP balances and MPT holdings. The protocol behaved correctly, but the result was easy to misread from an application-development perspective.

3. Multi-account Batch requirements are difficult to preflight

Field	Details
Category / Severity	Client Library / Documentation & Tutorials · High
Goal	Build a two-party atomic XRP-for-MPT settlement.
Expected	The SDK or a complete example explains the signers, authorization, submitter, and fee requirements.
Actual	We had to work out the requirements ourselves. An early submission returned <code>te1INSUF_FEE_P</code> ; settlement succeeded after we corrected the fee and signing configuration.
Reproduction	Build Batch → sign inner transactions → submit with insufficient fee → <code>te1INSUF_FEE_P</code> → correct fee → successful settlement
Impact	The interdependent requirements increase trial and error and the risk of failed settlement.
Improvement	Add preflight checks and publish a complete two-account signing and submission example.
Evidence	Failed response: <code>te1INSUF_FEE_P</code> · Successful Batch: <code>4D4686EA...0F9</code> · Fee: 60 drops

We expected the SDK to guide us through the signing and fee requirements, but we ended up working them out one by one. At first, it was difficult to tell whether the failure came from a missing signature, the submitter configuration, receiver authorization, or the fee. The `te1INSUF_FEE_P` response pointed us to the fee, but we still had to adjust the setup manually before the Batch succeeded. A complete two-account example would have saved us a lot of trial and error. Typically, the SDK supported the transaction types but we still had to build the “transaction objects” manually and we had to use low-level ledger queries.

Additional Observations

- **Custom Track 1 faucet : Documentation & Tutorials / Medium :**
The custom faucet did not match the usual `fundWallet` flow: it returned a newly funded wallet with a different response shape. A Track 1-specific request/response example would have made setup faster.
- **Vault ID helper discoverability : Client Library / Low :**
`hashes.hashVault` was not obvious from the vault-creation flow. **Improvement:** surface it directly in vault examples.
- **First-loss capital :** We figured that `tecINSUFFICIENT_FUNDS` didn't distinguish the difference between insufficient broker cover from insufficient vault liquidity. We can also say that we didn't test liquidation.